MAL4 Campaign Review & Approval -- High-Level Design

Project: Mobile Architecture Lab 4 (MAL4)
Track: MarTech -- Campaign Review & Approval
Document Identifier: MAL4-HLD-v1.0
Version: 1.0
Status: Draft -- Internal Review Pending
Preparation Date: 2026-11-01
Prepared By: Architecture & Engineering

Table of Contents

MAL4 Campaign Review & Approval — High-Level Design (HLD) 5

4.1 Document Control . . . 5
4.2 Executive Summary . . . 5
4.3 Introduction . . . 6
Purpose . . . 6
Scope . . . 6
Intended Audience . . . 6
Definitions and Terminology . . . 6
Assumptions . . . 7
Constraints . . . 7
Out-of-Scope Functionality . . . 7
4.4 System Context . . . 8
System Boundary . . . 8
External Actors / User Roles . . . 8
External Systems . . . 8
Trust Boundaries . . . 8
Diagram . . . 9
4.5 Architecture Drivers . . . 9
4.6 Architecture Style and Rationale . . . 10
How the Relevant Concepts Work Together (Not Competing) . . . 10
Comparison Matrix . . . 10
Selected Architecture . . . 11
Selection Rationale . . . 11
Rejected Alternatives . . . 11
Trade-offs and Consequences . . . 11
Diagram . . . 11
4.7 Technology Architecture . . . 11
4.8 Logical Architecture . . . 12
Diagram . . . 14
4.9 Module Decomposition . . . 15
Diagram . . . 16
4.10 Interface and Communication Architecture . . . 16
Module-to-Module Communication . . . 16
Interface Boundaries (High-Level — Exact Signatures in LLD) . . . 17
External Integration Boundaries . . . 17

Communication Failure Considerations *17*
4.11 Data Architecture 17
Major Business Entities and Relationships *17*
Data Ownership *18*
Persistence Strategy *18*
Data Lifecycle *18*
Data Consistency *18*
Versioning Approach *18*
Audit History *18*
Retention Considerations *18*
Limitations (Stated Explicitly) *18*
Diagram *19*
4.12 Workflow Architecture 20
Diagrams *22*
4.13 Security Architecture 23
Authentication Boundaries *23*
Authorization Model / User Roles *23*
Permission Enforcement *23*
Sensitive Operations *24*
Auditability *24*
Data Protection Considerations *24*
Explicit Limitation — Not Resolved Here *24*
Multi-Tenancy *24*
4.14 Reliability and Resilience Architecture 24
Failure Boundaries *24*
Persistence Failure Handling *24*
Concurrent Modification Risks *24*
Duplicate Operation Protection *24*
Recovery Strategy *24*
Notification Failure Handling *25*
Data Integrity *25*
4.15 Deployment and Runtime Architecture 25
Runtime Components (What Actually Runs Today) *25*
Local Persistence *25*
Remote Services *25*
External Dependencies *25*

Deployment Boundaries 25
The NFR-008/009 Gap, Stated Plainly 25
Multi-Device / Offline 26
Diagram 27
4.16 Quality Attribute Considerations 28
4.17 Architecture Constraints and Limitations 28
Accepted Trade-offs 28
Known Limitations 28
Scaling Boundaries 29
Future Evolution Possibilities 29
4.18 HLD Architecture Diagram Index 29
4.19 Requirement Traceability 30

MAL4 Campaign Review & Approval — High-Level Design (HLD)

4.1 Document Control

Field	Value
Document Title	MAL4 Campaign Review & Approval — High-Level Design
Project	Mobile Architecture Lab 4 (MAL4) — "Architects in AI Era"
Track	MarTech — Campaign Review & Approval
Document ID	MAL4-HLD-v1.0
Version	1.0
Status	Draft — pending team review
Author Role	Architecture & Engineering Lead
Creation Date	2026-11-01
HLD Reference	— (this is the root document)

Table 1

Revision History

Version	Date	Change
0.1	2026-11-01	Initial draft for internal review
1.0	2026-11-01	First complete release, covering system context through requirements traceability

Table 2

Approval Status

Reviewer	Role	Status	Date
—	Domain Research & Requirements Lead	Review pending	—
—	Architecture & Engineering Lead	Author	2026-11-01

Table 3

4.2 Executive Summary

The MAL4 Campaign Review & Approval system is a Flutter/Dart mobile application that replaces ad-hoc, email/spreadsheet-based campaign governance with a structured, auditable workflow: a Marketer creates a campaign, submits it for review, an authorized Reviewer approves, rejects, or requests changes, and only an approved campaign — still on its approved version — may be scheduled. This is required because unmanaged campaign governance causes brand, regulatory, budget, and audit risk, as established in the project's problem statement.

The proposed architectural direction is a **Modular Monolith**, organized **Feature-First** around a single `campaign` feature, with **Clean-Architecture-lite** internal layering (domain/application/infrastructure/presentation), an **MVVM** presentation pattern, and **narrow in-process domain events** for side-effect decoupling. No distributed infrastructure, no backend service, and no microservices are introduced — the architecture is sized to a 2-person team and a hackathon timeline (NFR-015).

Major architectural capabilities: a campaign lifecycle state machine enforced in the domain (never only in the UI); immutable, version-bound approvals that cannot go stale without detection; an append-only audit trail committed atomically with every state change; and role-based authorization checked before any mutation.

The most important design consideration, stated plainly rather than hidden, is that this architecture has **no backend process**. Several non-functional requirements (NFR-008/009) are phrased in literal client-server terms ("enforced server-side"). This document satisfies their *intent* — authorization is enforced in application/domain code, not merely hidden in the UI — but not a literal production security reading, because there is no server. This trade-off, and several others, are stated explicitly throughout this document rather than glossed over.

4.3 Introduction

Purpose

Define the complete system architecture for the MAL4 Campaign Review & Approval mobile application at a level sufficient for architectural review, defense before judges, and to serve as the stable basis from which the Low-Level Design (`MAL4-LLD.md`) derives module-internal detail. This document stays at the level of system decomposition, module interfaces, invocation relationships, and data/communication flow — it does not specify class methods, algorithms, or database field definitions; those belong to the LLD.

Scope

In scope: campaign creation/editing, submission for review, approve/reject/request-changes decisions, revision and re-submission, version management tied to approvals, the scheduling gate, audit trail, role-based access control, as defined in the project's approved scope boundaries.

Out of scope: campaign content creation tooling, actual campaign delivery/execution, audience segmentation engine, analytics/reporting, specific ad-platform/CRM/DAM integrations, A/B testing. These are acknowledged at the system boundary (§4.4) but not implemented.

Intended Audience

MAL4 judges and architecture reviewers, the project's requirements and domain-design team (for cross-reference against the workflow, data-design, and quality-attribute documentation), and the engineers who will carry this design into implementation.

Definitions and Terminology

Term	Definition
Campaign	The aggregate root entity representing a marketing campaign proposal and its lifecycle state
Campaign Version	An immutable snapshot of a campaign's content, created at submission or revision
Approval Record	An immutable record of a Reviewer's decision, bound to a specific campaign version
Audit Entry	An immutable, append-only record of a single lifecycle action
Material Change	Per FR-014's initial recommendation: any edit to an Approved campaign, triggering re-approval
Stale Review	A review decision attempted against a campaign version that is no longer current
Optimistic Concurrency	Conflict detection at write time via a version token, rather than locking at read time

Table 4

Assumptions

ID	Assumption	Confidence	Impact if Wrong
ASM-01	The domain/application layer plays the role NFR-008/009 describe as "server-side," enforced on-device rather than by a deployed server	Medium	If a real backend is required, the system boundary, persistence strategy, and security model change materially (§4.15)
ASM-02	Local-first Drift/SQLite persistence is sufficient for the MAL4 demo; multi-device sync is not required	Medium	If sync is required (OQ-016), persistence and concurrency design need a conflict-resolution layer not currently designed
ASM-03	Single-approval model (one Reviewer) is sufficient; multi-reviewer consensus (OQ-009) is not required	Medium	Adding consensus would require a new sub-state not currently in the lifecycle
ASM-04	Riverpod is an acceptable state-management choice for a 2-person team under a hackathon timeline	Low	Reversible — isolated to the Presentation layer by design

Table 5

Constraints

- Flutter/Dart only; no native platform-channel dependencies unless a requirement demands it.
- Team of 2 engineers, hackathon timeline.
- No distributed infrastructure (microservices, API gateways, Kubernetes, message brokers) — NFR-015: single-instance sufficient at 5–50 users.
- Single-tenant for the initial architecture (A-001; OQ-003 still open).

Out-of-Scope Functionality

Per the Scope subsection above: content creation tooling, delivery execution, audience segmentation, analytics/reporting, external platform integrations, A/B testing.

4.4 System Context

System Boundary

The boundary encloses exactly the Campaign Review & Approval capability: the campaign aggregate, its lifecycle/state machine, versioning, approval workflow, audit trail, and RBAC enforcement. Everything in the Out-of-Scope list sits outside this boundary and appears, at most, as a named external actor or port.

External Actors / User Roles

Actor	Role in System
Marketer (ST-001)	Creates, edits, submits, and revises campaigns. Cannot review own submissions (FR-018).
Reviewer (ST-002)	Approves, rejects, or requests changes on submitted campaigns. Modeled as a single role (A-003); Legal/Compliance and Finance stakeholders are specializations, not distinct actors, until multi-stage approval is in scope (OQ-001, open).
System Administrator (ST-003)	Manages roles/users and workflow configuration. Secondary to the core flow.

Table 6

External Systems

External System	Direction	Treatment
Identity Provider	Inbound	Abstracted behind an `AuthProvider` port (§4.13). Not implemented for the MAL4 deliverable.
Notification Service	Outbound	Abstracted behind a `NotificationGateway` port. Failures here never block the core state transition (NFR-005).
Scheduling System	Outbound	Out of scope to implement; the boundary ends at enforcing the scheduling *gate* (FR-012).
Asset/Content Storage	Bidirectional	Out of scope (A-004: content is referenced, not stored).
Analytics/Reporting	Outbound	Out of scope; acknowledged at the boundary only.

Table 7

Trust Boundaries

The most important trust boundary in this system is **not** between "client" and "server" — there is no server. It is between the Presentation layer (UI, which must never be trusted to enforce anything) and the Application/Domain layers (which perform every authorization and business-rule check). See §4.13 for the full security-architecture

treatment of this distinction.

Diagram

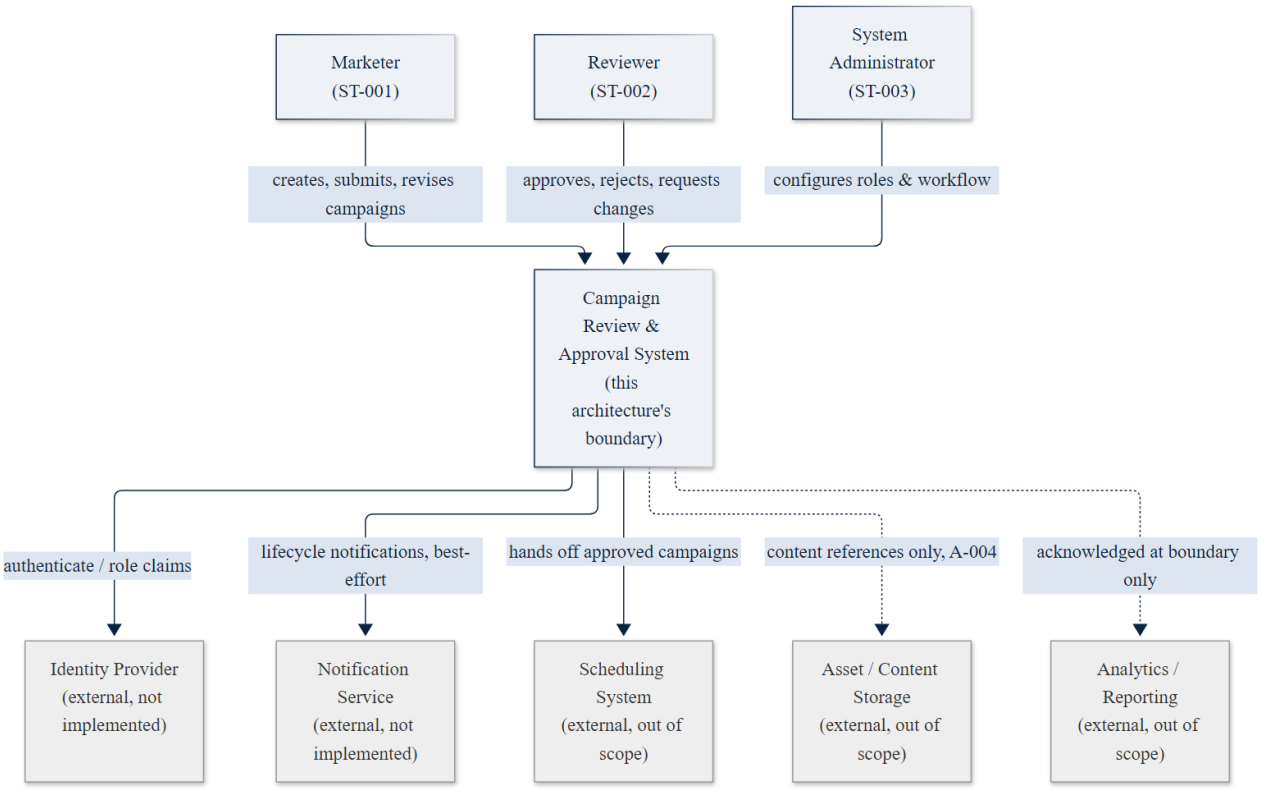


Figure 1: *System Context Diagram*

4.5 Architecture Drivers

Driver	Source	Architectural Response
Lifecycle rules must be testable without a UI/DB	MAL4 rubric: Testing & Observability (15%)	Domain layer has zero outward dependencies (§4.8)
Approval must be bound to a specific version	NFR-002, FR-007	Version token + `ApprovalRecord.versionReviewed` (§4.11, full mechanism in LLD)
Concurrent edits must not silently corrupt state	NFR-001, NFR-003	Optimistic concurrency (§4.11)
Every lifecycle action must be auditable	FR-015, NFR-012	Append-only `AuditEntry`, same transaction as the state change (§4.11)
Notification failure must not block the workflow	NFR-005	In-process domain events, side effects decoupled from the write path (§4.10)
RBAC and self-review prevention	FR-017/018, NFR-008/009/010	Application + Domain layer checks, never UI-only (§4.13)
No distributed infrastructure disproportionate to team/timeline	NFR-015; 2-person team constraint	Modular Monolith, local-first persistence (§4.6, §4.15)

Driver	Source	Architectural Response
Hackathon deliverable requires a working mobile user journey	MAL4 rubric: Working User Journey (15%)	Flutter/Dart target platform; architecture kept proportionate rather than theoretical (§4.6)

Table 8

Key architectural risks (full register in §4.17): judges reading NFR-008/009 literally and scoring the local-first design as non-compliant; 2-person documentation/implementation bandwidth; documentation/diagram drift once code exists; the conceptual ER diagram being mistaken for the physical schema.

4.6 Architecture Style and Rationale

How the Relevant Concepts Work Together (Not Competing)

Layered Architecture, Clean Architecture, Feature-First Architecture, MVVM, Repository Pattern, and Event-Driven Architecture are not five competing choices to pick one from — each answers a different architectural question:

Level	Question Answered	Applied Here As
Deployment/codebase boundary	How many deployable units?	Modular Monolith
Code organization	Feature or technical-layer partitioning?	Feature-First
Internal dependency direction	Which direction do dependencies point, within the feature?	Clean-Architecture-lite (not Layered, not full Clean)
Presentation-internal pattern	How does UI bind to state?	MVVM
Cross-layer seam	How is persistence substituted without touching business logic?	Repository Pattern
Side-effect communication	Direct calls or events?	Narrow, in-process Event-Driven

Table 9

Comparison Matrix

Scored High/Medium/Low against this project's actual constraints (2-person team, hackathon timeline, Flutter/Dart, 5–50 users / 10–200 active campaigns per NFR-015):

Criterion	Layered (classic N-tier)	Clean Architecture (full)	Feature-First	MVVM	Modular Monolith	Event-Driven (full EDA)
Separation of concerns	Medium	High	High	Medium (presentation only)	N/A	High, orthogonal
Complexity	Low	High	Low–Medium	Low	Low	High if applied broadly
Testability	Medium	High	Medium	High	N/A	Medium
Maintainability	Low as features grow	High, at ceremony cost	High	High	High	Low if overused
Scalability	Low relevance	N/A	N/A	N/A	Matches NFR-015	Over-engineered here

Criterion	Layered (classic N-tier)	Clean Architecture (full)	Feature-First	MVVM	Modular Monolith	Event-Driven (full EDA)
Development speed (2-person)	High	Low	High	High	High	Low if applied broadly
Suitability for Flutter	Medium	Medium	High	High	High	Low
Suitability for MAL4 requirements	Medium	High on paper, costly in practice	High	High	High	Low as dominant style

Table 10

Selected Architecture

Modular Monolith + **Feature-First** + **Clean-Architecture-lite** + **MVVM** + **Repository Pattern** (as the dependency-inversion seam) + **narrow in-process Event-Driven** communication.

Selection Rationale

Matches the explicit constraint to avoid distributed infrastructure while isolating domain invariants from UI/infra. Full Clean Architecture's ceremony is disproportionate to one feature and two developers — the *principle* (dependency inversion, domain isolation) is kept, the *ceremony* is not. MVVM is the natural shape for Riverpod/Bloc-style notifiers and operates one level below the Clean-lite boundary, not in competition with it.

Rejected Alternatives

- **Full Clean Architecture** — too much ceremony for a single aggregate and 2 developers.
- **Classic Layered (global UI/BLL/DAL)** — partitions by technical type across the whole app, recoupling otherwise-independent features and removing the seam needed for swappable persistence.
- **Full Event-Driven Architecture** — appropriate for multi-service systems with independent scaling needs, which this explicitly is not.

Trade-offs and Consequences

Some boilerplate (use-case classes, repository interfaces) versus "call the database from the widget" — accepted because it makes the domain testable in isolation. The "lite" Clean Architecture risks boundary ambiguity without a strict reference implementation — mitigated by this document's explicit layer definitions (§4.8) and the LLD's concrete package structure.

Diagram

Referenced again in §4.18's Diagram Index; no additional diagram beyond §4.8's Logical Architecture Diagram is needed for style selection itself.

4.7 Technology Architecture

Concern	Selection	Architectural Purpose	Alternatives Considered	Status
Mobile framework	Flutter	Fixed by the MAL4 challenge (mobile-first requirement)	None — not evaluated, not a free choice	Fixed
Programming language	Dart	Follows from Flutter	None	Fixed
State management	Riverpod	Compile-time-safer DI than Provider; less ceremony than Bloc for this scale; async-notifier pattern maps cleanly onto `Result`-returning use cases	Bloc/Cubit, Provider	**PROPOSED** — not yet implemented or finalized by the team
Navigation	GoRouter (optional) or plain `Navigator`	Declarative routing if deep-linking (review queue → detail → back) is justified	Plain `Navigator` 2.0	**OPEN DECISION** — not yet justified either way
Persistence	Drift (over SQLite)	Type-safe queries, migrations, transactions, reactive streams; relational joins fit campaign→version→approval→audit better than an object DB	Raw SQLite, Isar, backend API + DB	**PROPOSED**, with explicit limitations stated in §4.11/§4.15
Backend integration	**None**	No backend exists or is planned for this deliverable	A real backend (would resolve NFR-008/009)	**Explicitly not built** — stated, not invented
Communication protocol	None (in-process only)	No network calls in the core workflow	—	N/A
Testing infrastructure	`flutter_test` + `mocktail` + Drift in-memory DB	Satisfies MAL4's Testing & Observability criterion (15%)	—	**PROPOSED**
Development tooling	Dart analyzer / lint rules (recommended, not yet configured)	Enforce the layer dependency-direction rule at compile/lint time	—	**OPEN DECISION** — not yet set up; no code exists yet

Table 11

No technology above is claimed as finalized by team approval — each is marked PROPOSED or OPEN DECISION to reflect its actual status.

4.8 Logical Architecture

Dependencies point inward only: **Presentation** → **Application** → **Domain** ← **Infrastructure.** Domain has zero outward dependencies — no Flutter, no database driver, no HTTP client.

Layer	Responsibility	Scope / Owned Concerns	Inputs	Outputs	Dependencies	Dependency Direction
Domain	Lifecycle state machine, versioning, approval-binding, self-review prevention	`Campaign` aggregate, `CampaignVersion`, `ApprovalRecord`, `AuditEntry`	Method calls from Application	New aggregate state, domain events	None	Zero outward dependencies
Application	Orchestrate use cases: authorize, concurrency-check, delegate to domain, persist, emit events	One use case per FR-numbered operation	Commands from Presentation	`Result<Campaign, DomainError>`	Domain, repository/adapter interfaces (owned here)	Depends on Domain only
Infrastructure	Implement domain-owned interfaces against real technology	Repositories (Drift), `AuthProvider`, `NotificationGateway` adapters	Calls from Application (via interfaces)	Persisted state, delivered (or logged-failed) notifications	Domain (to implement its interfaces), external systems	Implements interfaces defined by Application/Domain; never depended on by them
Presentation	UI, state projection, user input	Screens, widgets, navigation, UI-local state	User actions	Commands to Application	Application only	Depends on Application only; never on Domain or Infrastructure directly

Table 12

A widget can only issue a command to a use case; it cannot mutate a `Campaign` directly — this is what keeps lifecycle/approval logic testable without a UI.

Diagram

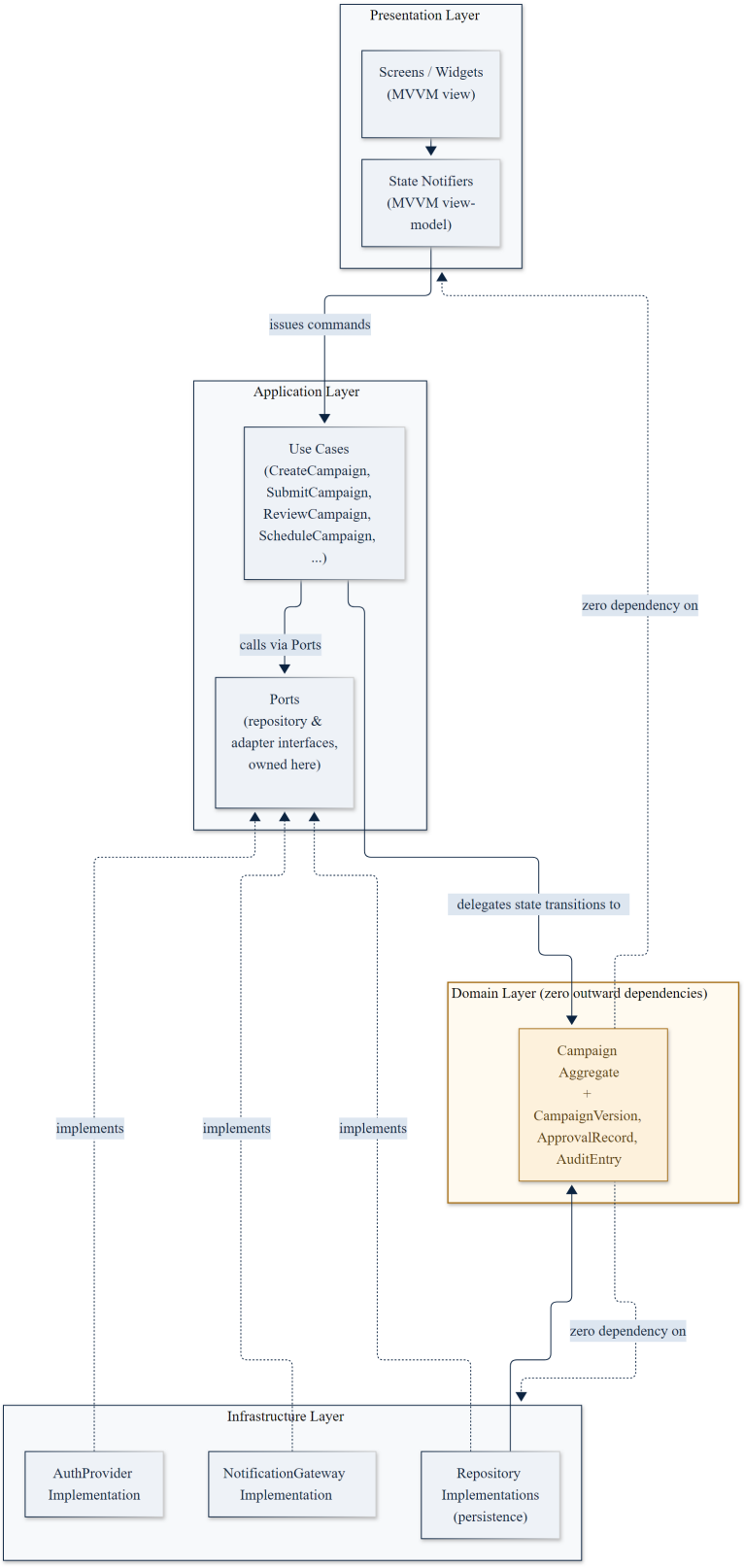


Figure 2: *Logical Architecture Diagram*

4.9 Module Decomposition

Design decision: Review and Approval are modeled as a single module, **Review & Approval Workflow**, rather than two — per FR-006–FR-011, a Reviewer's decision *is* the approval workflow; splitting them would create an artificial seam with no independent data or rule of its own.

All modules are packaged as Dart libraries/folders inside a single `campaign` feature module (Modular Monolith + Feature-First) — not separate Flutter packages or services (full package tree: LLD §5.2.B / package architecture).

Module	Purpose	Exposed Interfaces	Consuming Modules	Dependencies	Data Ownership	Architectural Constraints
Authentication & Authorization	Identify actor/role; gate every state-transitioning use case	`AuthProvider`	All other modules (cross-cutting)	External Identity Provider (not implemented)	None persistent in-app	Checked in Application layer, never only UI
Campaign Management	Create/edit campaigns in Draft/Changes Requested	`CreateCampaign`, `EditCampaign` use cases	Presentation	Campaign Version Management, Auth	`Campaign` entity (metadata, state, version pointer)	Edit on wrong state rejected at domain layer
Campaign Version Management	Manage immutable version snapshots + concurrency token	Exposed via `Campaign` aggregate methods	Campaign Management, Review & Approval Workflow, Scheduling	Shares the `Campaign` aggregate boundary	Version history	Versions created only at submit/revise (FR-013)
Review & Approval Workflow (merged)	Record approve/reject/request-changes; invalidate on material change	`ReviewCampaign`, `ReviseCampaign` use cases	Presentation	Version Management, Auth, Audit	`ApprovalRecord`	Stale-version review rejected; self-review rejected
Audit Management	Immutable, queryable log of every lifecycle action	`AuditLogRepository`	All action-performing modules	Local persistence	Audit log (same transaction as state change)	Append-only; no update/delete interface
Notification Management	Best-effort delivery, decoupled from write path	`NotificationGateway`	Review & Approval Workflow, Campaign Management	External Notification Service (not implemented)	None in scope (outbox: open)	Never blocks the workflow (NFR-005)
Scheduling	Gate-only transition to Scheduled	`ScheduleCampaign` use case	Presentation	Version Management, Audit	None new	Duplicate-schedule attempt rejected

Table 13

Diagram

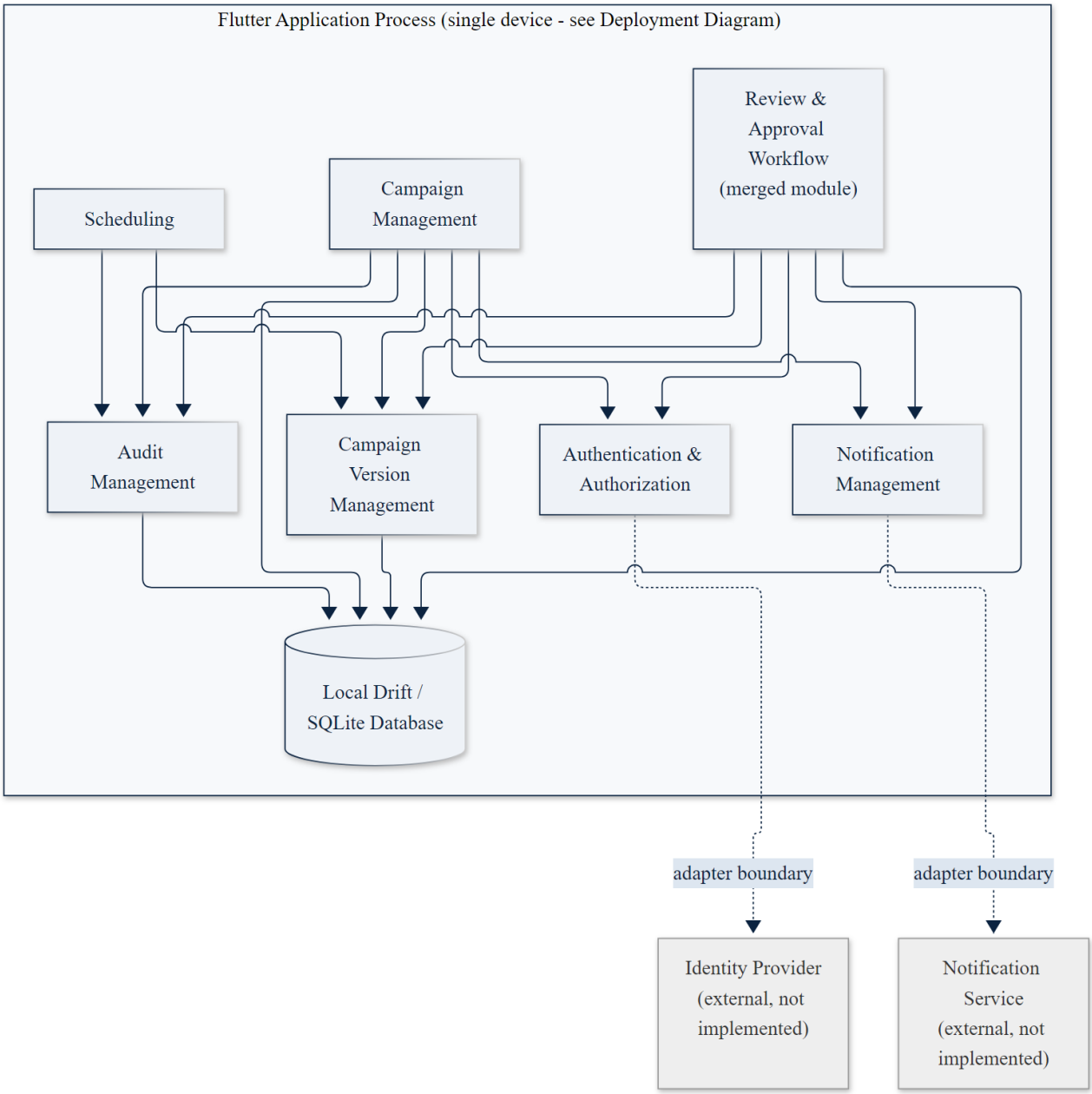


Figure 3: *Component Architecture Diagram*

This diagram and table describe module-level interfaces and interactions only — exact class methods and internal algorithms are covered in the Low-Level Design (`MAL4-LLD.md` §5.2–§6).

4.10 Interface and Communication Architecture

Module-to-Module Communication

All communication within the application is **synchronous and in-process** — there are no asynchronous inter-module calls in the core write path. The one deliberate exception is side-effect dispatch (below).

Interaction	Style	Rationale
Presentation → Application	Synchronous command dispatch	Presentation issues a command object (e.g., "submit this campaign"); it does not call Domain or Infrastructure directly
Application → Domain	Synchronous method delegation	The use case calls the aggregate's method and receives a new aggregate state or a thrown domain error, in the same call
Application → Infrastructure (via Ports)	Synchronous (persistence), asynchronous-fire-and-forget (notifications)	Repository calls are synchronous and awaited (the use case needs the result to respond to the user); notification dispatch is asynchronous and the use case does not wait on it (NFR-005)
Application → Side effects (notifications)	**Asynchronous, decoupled via in-process domain events**	Justified explicitly: a notification failure must never block or roll back an already-committed state transition

Table 14

Interface Boundaries (High-Level — Exact Signatures in LLD)

Boundary	Input Responsibility	Output Responsibility
Presentation → Application	Well-formed command objects only (field validation already done at this boundary where applicable)	`Result` success/failure, mapped to UI state
Application → Domain	The aggregate's own current state (always freshly loaded, never a stale cache)	New aggregate state or a domain error
Application → Repository	The aggregate plus the version it believes is current (`expectedVersion`)	Success, or a concurrency-conflict signal

Table 15

External Integration Boundaries

Identity Provider and Notification Service are the only external integration boundaries, both abstracted behind ports (`AuthProvider`, `NotificationGateway`) with no live implementation for this deliverable (§4.15).

Communication Failure Considerations

A failure in the Application → Repository call surfaces as a typed error to the user (no silent retry at this layer). A failure in the side-effect dispatch to Notification Management is logged and retried independently — it is structurally incapable of affecting the result already returned to the user, because it happens strictly after that result is determined.

4.11 Data Architecture

Major Business Entities and Relationships

`Campaign` (1) — owns — (many) `CampaignVersion`; `CampaignVersion` (1) — may have — (0..1) `ApprovalRecord`; `Campaign` (1) — produces — (many) `AuditEntry`.

Data Ownership

Data	Owner (module)	Mutability
`Campaign` metadata + state + version pointer	Campaign Management	Mutable only in Draft/Changes Requested
`CampaignVersion`	Campaign Version Management	Immutable once created
`ApprovalRecord`	Review & Approval Workflow	Immutable; `supersededBy` set, never deleted
`AuditEntry`	Audit Management	Append-only, no update/delete

Table 16

Persistence Strategy

Local-first, Drift over SQLite. Evaluated against raw SQLite (no type safety), Isar (weaker fit for the relational joins this domain needs), and a backend API + database (disproportionate to a 2-person hackathon timeline and not confirmed as required by any FR). Campaign-state and audit-entry writes commit in one local transaction.

Data Lifecycle

`Create (Draft, unversioned working copy)` → `Submit (CampaignVersion 1, Pending Review)` → `Review decision (ApprovalRecord bound to version)` → `{Approved | Rejected | Changes Requested}` → `[Revise → new CampaignVersion → Pending Review]*` → `Schedule (terminal)`. A material change to an Approved campaign re-enters this cycle at Draft, with the prior `ApprovalRecord` superseded, not deleted (FR-014).

Data Consistency

Optimistic concurrency via a `currentVersion` token (NFR-001/003) is the sole consistency mechanism — no pessimistic locking, no distributed transaction coordinator. Full mechanism: `MAL4-LLD.md` §6 (Concurrent Modification).

Versioning Approach

Exactly two version-creation trigger points (FR-013): submission and revision-resubmission. Versions are immutable, full snapshots (A-010), never diffs.

Audit History

Append-only, queryable by campaign/actor/time range (FR-015), supporting full reconstruction of a campaign's approval history (FR-016, NFR-012).

Retention Considerations

Not yet confirmed — NFR-013's retention periods are provisional pending the quality-attributes documentation, which has not yet been finalized. No retention/purge mechanism is implemented.

Limitations (Stated Explicitly)

Local-only persistence does not provide cross-device synchronization; on-device enforcement is not equivalent to server-side enforcement (NFR-008/009). See §4.15 for the full statement.

Diagram

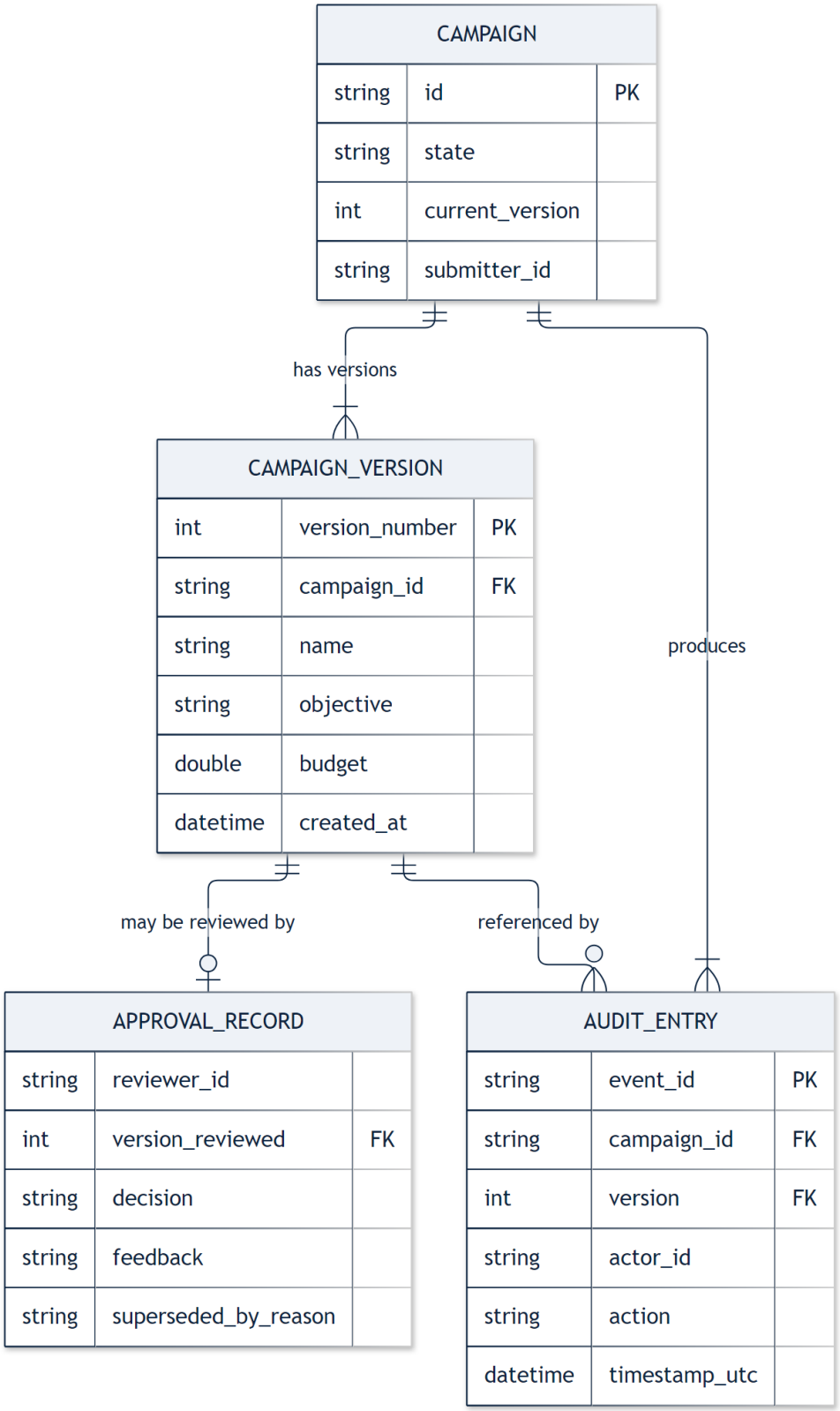


Figure 4: *High-Level Entity Relationship Diagram*

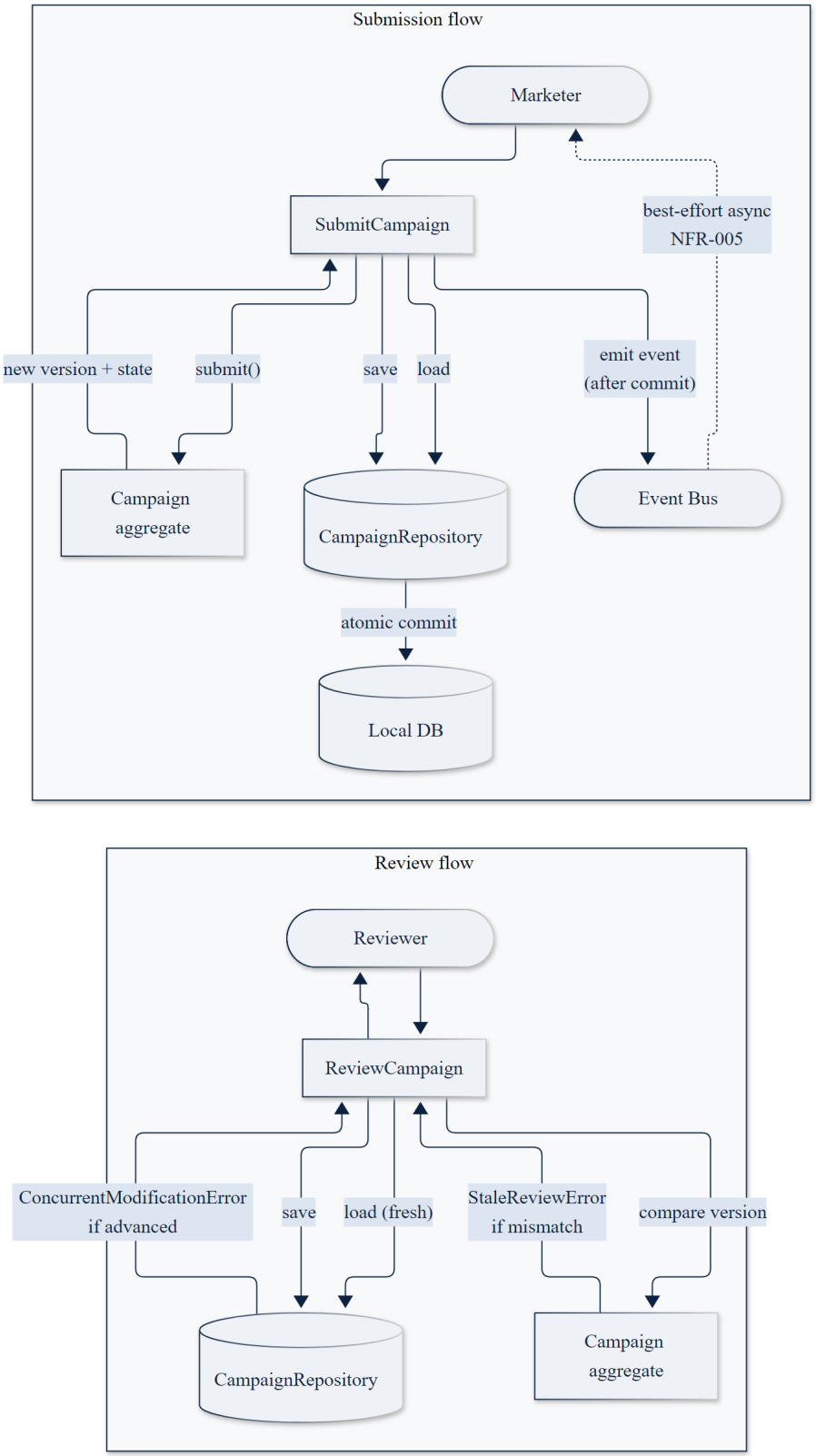


Figure 5: *Data Flow Diagram*

4.12 Workflow Architecture

The campaign lifecycle, at the workflow level (full state-transition table with guards: `MAL4-LLD.md` §6):

Creation — Marketer creates a campaign → validated → Draft (FR-001).

Submission — Draft/Changes Requested → creates a `CampaignVersion`, transitions to Pending Review (FR-004).

Review — Pending Review → self-review and current-version checks → `ApprovalRecord` recorded → Approved/Rejected/Changes Requested (FR-006–FR-009).

Change Requests — Reviewer requests changes with required feedback → Changes Requested (FR-009).

Resubmission — Marketer revises a Changes-Requested campaign → new `CampaignVersion` → back to Pending Review (FR-010).

Scheduling — Approved, version re-verified as still current → Scheduled (FR-012, NFR-007).

Material-change flow — an edit to an Approved campaign invalidates the existing `ApprovalRecord` (superseded, not deleted) and returns the campaign to Draft (FR-014).

Diagrams

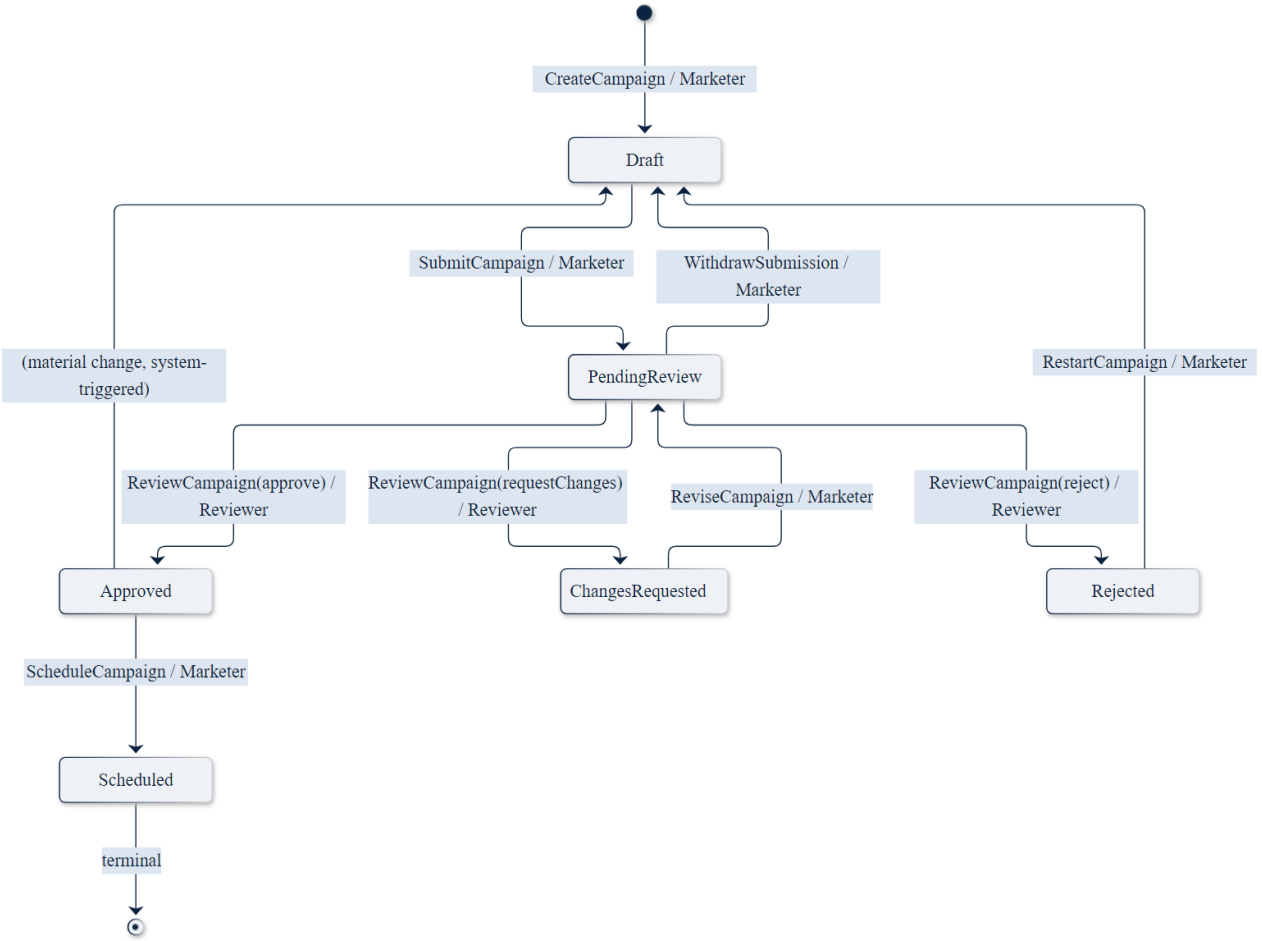


Figure 6: *Campaign State Machine Diagram*

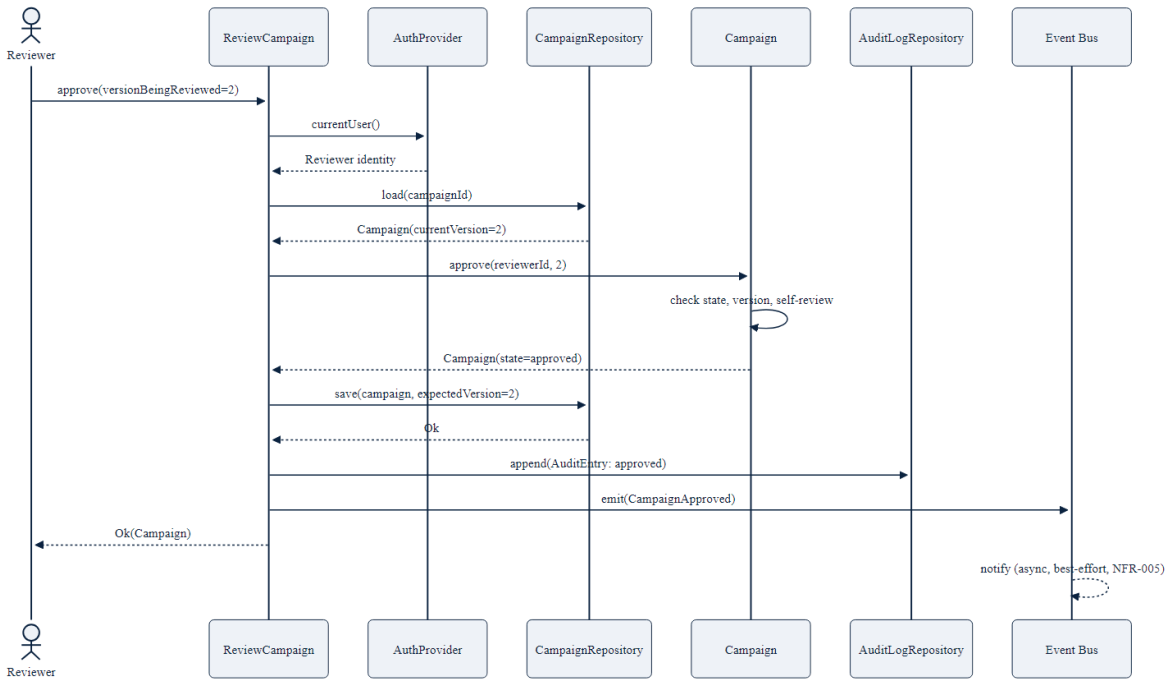


Figure 7: *Approval Sequence Diagram*

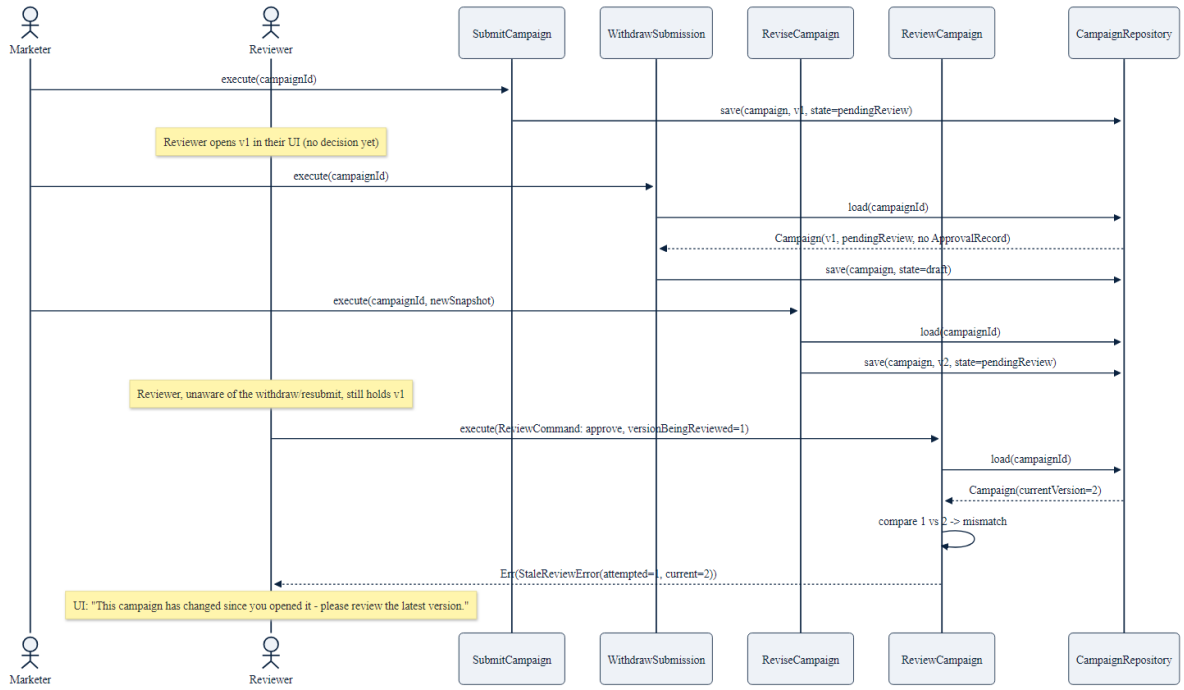


Figure 8: *Revision Sequence Diagram*

shown here for workflow continuity; the full sequence-level detail, including class and method references, is in the Low-Level Design

---

4.13 Security Architecture

Authentication Boundaries

Every use case requires an `AuthenticatedUser{id, role}` supplied by `AuthProvider`. An unauthenticated request is rejected before reaching any use case (NFR-008).

Authorization Model / User Roles

Role	Permitted Actions
Marketer	Create, edit (Draft/Changes Requested), submit, withdraw, view own campaigns
Reviewer	View submitted campaigns, approve, reject, request changes
Admin	All of the above, plus manage users, configure workflows, view audit logs, view all campaigns

Table 17

Permission Enforcement

Checked in the Application layer **before** any domain mutation, and defensively again inside the `Campaign` aggregate for self-review specifically (defense in depth) — never relying on the UI hiding a button.

Sensitive Operations

Approval/rejection/schedule decisions are the most sensitive operations (financially/legally consequential per the problem statement's Impact table). Each is role-gated, self-review-gated, version-gated, and unconditionally audited.

Auditability

The audit log is append-only by construction — no update/delete interface exists, not merely a policy convention.

Data Protection Considerations

No sensitive data classification (PII, payment data) is defined in the approved requirements documentation beyond general campaign business data; nothing beyond what's already stated is assumed.

Explicit Limitation — Not Resolved Here

NFR-008/NFR-009 describe server-side enforcement; this architecture has no server. See §4.15 for the full statement.

Multi-Tenancy

Not addressed (NFR-011 conditional on OQ-003, open). Single-tenant only.

4.14 Reliability and Resilience Architecture

Failure Boundaries

Detection is local and synchronous: invalid transitions are caught inside the `Campaign` aggregate before any persistence call; stale versions and concurrency conflicts are caught at the repository's write boundary.

Persistence Failure Handling

A campaign-state write and its audit entry commit together or not at all (one local transaction) — no explicit application-level recovery routine is required beyond this, because there is no partial state possible to repair.

Concurrent Modification Risks

Addressed via optimistic concurrency (a version token compared at write time) — not pessimistic locking, which would be disproportionate to this scale. Full behavior: `MAL4-LLD.md` §6.

Duplicate Operation Protection

A second identical action is rejected because the first already moved the campaign out of the state the second requires. This is a side effect of the concurrency mechanism, not a separate safeguard.

Recovery Strategy

SQLite's WAL-mode durability means a committed transaction survives an app crash; an uncommitted one leaves no trace — recovery is a property of the persistence engine, not

a mechanism this architecture builds separately.

Notification Failure Handling

Logged and retried asynchronously, independent of the write path (NFR-005) — never surfaced as workflow-blocking.

Data Integrity

Guaranteed only within a single local database — this does not extend across devices (§4.15's limitation).

Note on scope: the detailed, scenario-by-scenario failure table (nine scenarios with detection, recovery, and testing-approach columns) is implementation-level detail and is maintained in the Low-Level Design, `MAL4-LLD.md` §6, rather than duplicated here.

4.15 Deployment and Runtime Architecture

Runtime Components (What Actually Runs Today)

A single Flutter application process on a single mobile device: Presentation, Application, Domain, and Infrastructure layers all execute in-process, on-device.

Local Persistence

A local SQLite database file (via Drift), on the same device.

Remote Services

None exist. `AuthProvider` and `NotificationGateway` are architectural seams (interfaces) with no live external implementation for the MAL4 deliverable — not invented infrastructure, documented as not-yet-implemented adapters.

External Dependencies

None at runtime (no network dependency in the core workflow — the app is offline-capable by construction).

Deployment Boundaries

Component	Runs Where	Status
UI, use cases, domain, repositories	On-device, in-process	Implemented by this architecture
`AuthProvider` → real Identity Provider	Would run remotely, if a backend existed	Not implemented; interface only
`NotificationGateway` → real Notification Service	Would run remotely, if a backend existed	Not implemented; interface only
Scheduling System (actual delivery execution)	Out of scope entirely	Not implemented; boundary only

Table 18

The NFR-008/009 Gap, Stated Plainly

NFR-008 (401 on unauthenticated requests) and NFR-009 ("enforced server-side, not only in the client") describe a client-server deployment. **This deployment has no**

server. The architecture satisfies the intent — authorization is enforced in Application/Domain layers, not merely hidden in the UI — but cannot satisfy the literal requirement, because there is no process boundary to enforce anything across. This is an accepted limitation, not resolved by inventing a backend that is not part of this deliverable.

Multi-Device / Offline

Offline-capable by construction; explicitly **not** multi-device-capable (OQ-016, open).

Diagram

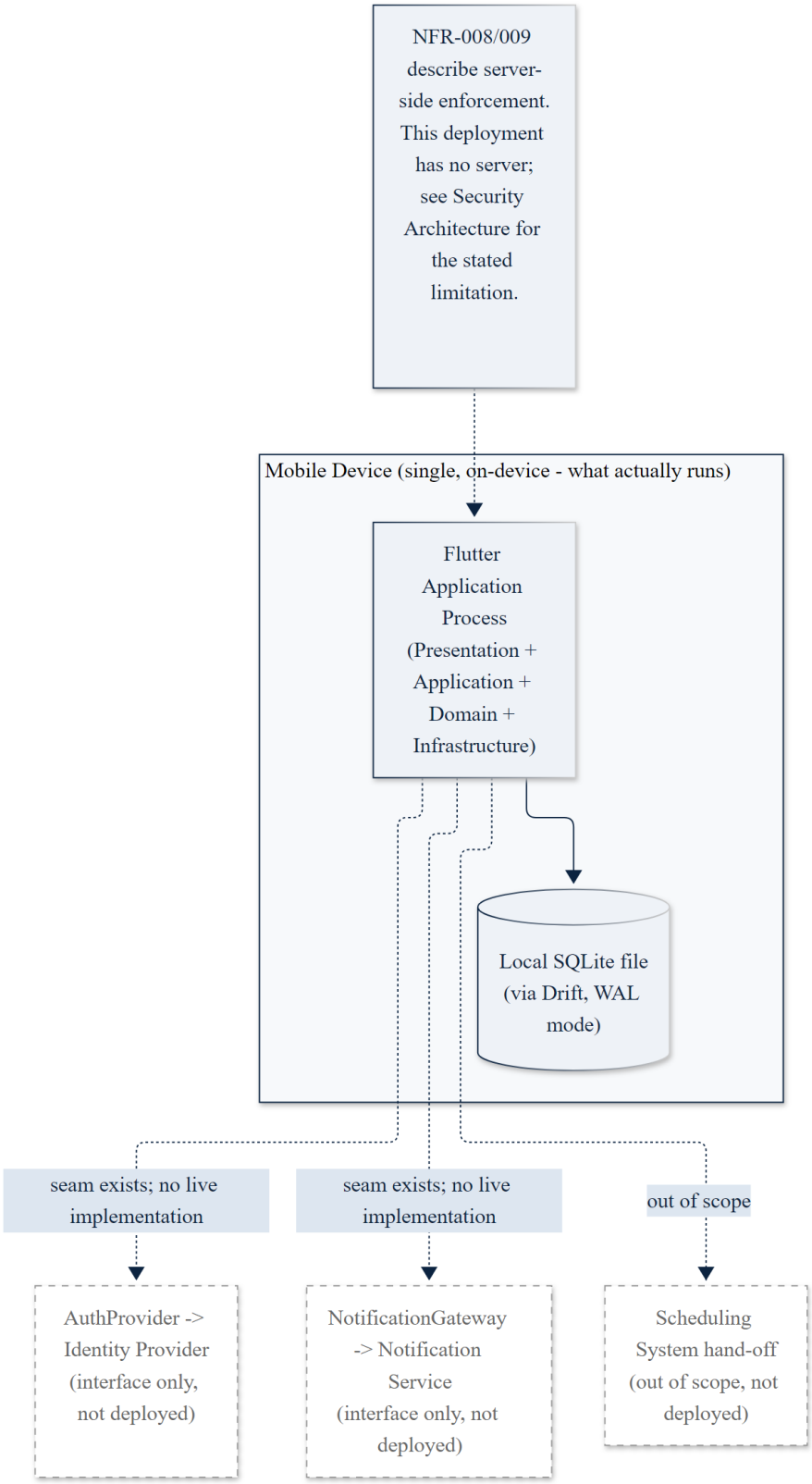


Figure 9: *Deployment Diagram*

4.16 Quality Attribute Considerations

Quality Attribute	Architecture Decision Connecting to It	Measurable Target (Only Where Supported)
Performance	Local-only reads/writes, no network round trip in the core workflow	NFR-014's provisional targets (e.g., <1s for review decisions) — provisional, not independently verified
Reliability	Atomic state+audit commit; WAL-mode durability	NFR-006's provisional availability targets — not verified; this is a single-device app, not a hosted service, so classic uptime metrics don't directly apply
Maintainability	Clean-lite layering, dependency inversion, Feature-First organization	No numeric target — qualitative, judged by whether persistence/tech choices can change without touching the domain
Testability	Domain has zero outward dependencies	MAL4 rubric: Testing & Observability, 15% — test scenarios cataloged in LLD §5.2.K, not yet executed (no code exists)
Security	RBAC + self-review checks in Application/Domain, never UI-only	NFR-008/009/010 — partially satisfied; the server-side gap is explicit (§4.15), not claimed as met
Extensibility	Repository/port interfaces as swap points; module boundaries aligned to FR clusters	Qualitative — supports adding multi-stage approval or a real backend without rewriting the domain (§4.17)
Usability	Distinct, specific error messages per failure type (never a generic "something went wrong")	Qualitative — no usability testing has occurred

Table 19

No target above is reported as achieved through testing; all are either provisional (inherited from the approved non-functional requirements) or qualitative architectural properties.

4.17 Architecture Constraints and Limitations

Accepted Trade-offs

Some use-case/repository boilerplate for testability; Clean-lite over full Clean Architecture; optimistic over pessimistic concurrency; local-first over a synced/backend architecture.

Known Limitations

NFR-008/009 server-side-enforcement gap — the single most likely point of challenge from judges reading the NFRs literally.

Idempotency-key scheme (NFR-004) is undesigned — a true race (not a retry-after-success) is not fully covered by optimistic concurrency alone.

Multi-reviewer consensus (OQ-009) is unresolved.

Offline/multi-device sync (OQ-016) is unresolved — offline-capable, but not multi-device-capable.

Notification outbox/retry persistence is undesigned.

No Flutter/Dart code exists — every signature and pseudocode block in the Low-Level Design is a design artifact, not implementation evidence. Nothing in either document should be read as already built or tested.

The workflow, data-design, and quality-attribute documentation owned by the requirements team is not yet finalized — several cross-references in this document point to specifications that are still in progress.

Scaling Boundaries

Sized for 5–50 users, 10–200 active campaigns (NFR-015, A-005). Not load-tested; this is an architecture-defense discussion point, not a benchmarked claim.

Future Evolution Possibilities

Persistence can be swapped (local Drift → backend-synced) without touching the domain, via the repository interfaces. Multi-stage approval can be added inside the Review & Approval Workflow module without restructuring other modules. A real backend can be introduced behind the existing `AuthProvider`/`NotificationGateway` interfaces without rewriting use cases.

4.18 HLD Architecture Diagram Index

Figure	Name	Purpose	Section
Figure 1	System Context Diagram	Actors, external systems, system boundary	§4.4
Figure 2	Logical Architecture Diagram	Layers and dependency direction	§4.8
Figure 3	Component Architecture Diagram	Module decomposition and interactions	§4.9
Figure 4	Entity Relationship Diagram (conceptual)	Domain-level entity relationships, not the physical schema	§4.11
Figure 5	Data Flow Diagram	How a command flows through layers to persistence and back	§4.11
Figure 6	Campaign State Machine Diagram	Lifecycle states and transitions	§4.12
Figure 7	Approval Sequence Diagram	Reviewer decision flow, including the version and self-review checks	§4.12
Figure 8	Revision Sequence Diagram	Withdraw-and-resubmit flow underlying the stale-review scenario	§4.12
Figure 9	Deployment Diagram	Runtime components, local-only, no backend	§4.15

Table 20

4.19 Requirement Traceability

Condensed view (Requirement → Architecture Component → HLD Section → Validation Method). The complete 7-column matrix, including LLD section, diagram, and test-scenario columns, is maintained in `architecture-traceability.md`.

Requirement	Architecture Component	HLD Section	Validation Method
FR-001–003	Campaign Management module	§4.9	Unit tests on the `Campaign` aggregate (LLD §5.2.K)
FR-004, FR-005	Submit/Withdraw use cases	§4.9, §4.12	Use-case tests with a fake repository
FR-006–FR-011	Review & Approval Workflow module	§4.9, §4.12	Integration test: stale-approval scenario
FR-012	Scheduling module	§4.9, §4.12	Concurrency test: duplicate-schedule rejection
FR-013, FR-014	Campaign Version Management module	§4.11, §4.12	Unit test: version-token conflict; material-change invalidation
FR-015, FR-016	Audit Management module	§4.11, §4.13	Repository test: append-only, no mutation path
FR-017, FR-018	Authentication & Authorization module	§4.13	Authorization-check unit tests
FR-019	Notification Management module	§4.9, §4.14	Use-case test: notification failure doesn't roll back the transaction
NFR-001–003	Repository optimistic-concurrency check	§4.11, §4.14	Concurrency test: `ConcurrentModificationError` on stale write
NFR-004	Not yet designed	§4.17 (flagged)	**Not yet validated — open item**
NFR-005	Domain event + Notification module	§4.10, §4.14	Use-case test: commit independent of notification outcome
NFR-007	Scheduling module	§4.9, §4.14	Concurrent-schedule test
NFR-008–010	Authentication & Authorization module	§4.13	Authorization-rejection tests
NFR-012, NFR-013	Audit Management module	§4.13, §4.11	Atomic audit/state commit test
NFR-015	Modular Monolith decision	§4.6	Not load-tested — architecture-defense discussion only

Table 21